main.py

Visualize Run ▶

```python
regions = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']
neighbours = {
    'WA': ['NT', 'SA'],
    'NT': ['WA', 'SA', 'Q'],
    'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],
    'Q': ['NT', 'SA', 'NSW'],
    'NSW': ['Q', 'SA', 'V'],
    'V': ['SA', 'NSW'],
    'T': [],
}
colours = ['Red', 'Green', 'Blue']

def consistent(region, colour, assignment):
    return all(assignment.get(nb) != colour for nb in neighbours[region])

def backtrack(assignment):
    if len(assignment) == len(regions):
        return assignment
    region = next(r for r in regions if r not in assignment)
    for colour in colours:
        if consistent(region, colour, assignment):
            assignment[region] = colour
            result = backtrack(assignment)
            if result:
                return result
            del assignment[region]
```

main.py

Visualize

Run ▶

```
    'V': ['SA', 'NSW'],
    'T': [],
}
colours = ['Red', 'Green', 'Blue']

def consistent(region, colour, assignment):
    return all(assignment.get(nb) != colour for nb in neighbours[region])

def backtrack(assignment):
    if len(assignment) == len(regions):
        return assignment
    region = next(r for r in regions if r not in assignment)
    for colour in colours:
        if consistent(region, colour, assignment):
            assignment[region] = colour
            result = backtrack(assignment)
            if result:
                return result
            del assignment[region]
    return None

solution = backtrack({})
for region, colour in solution.items():
    print("%-4s -> %s" % (region, colour))
```

Output

```
WA   -> Red
NT   -> Green
SA   -> Blue
Q    -> Red
NSW  -> Green
V    -> Red
T    -> Red
```